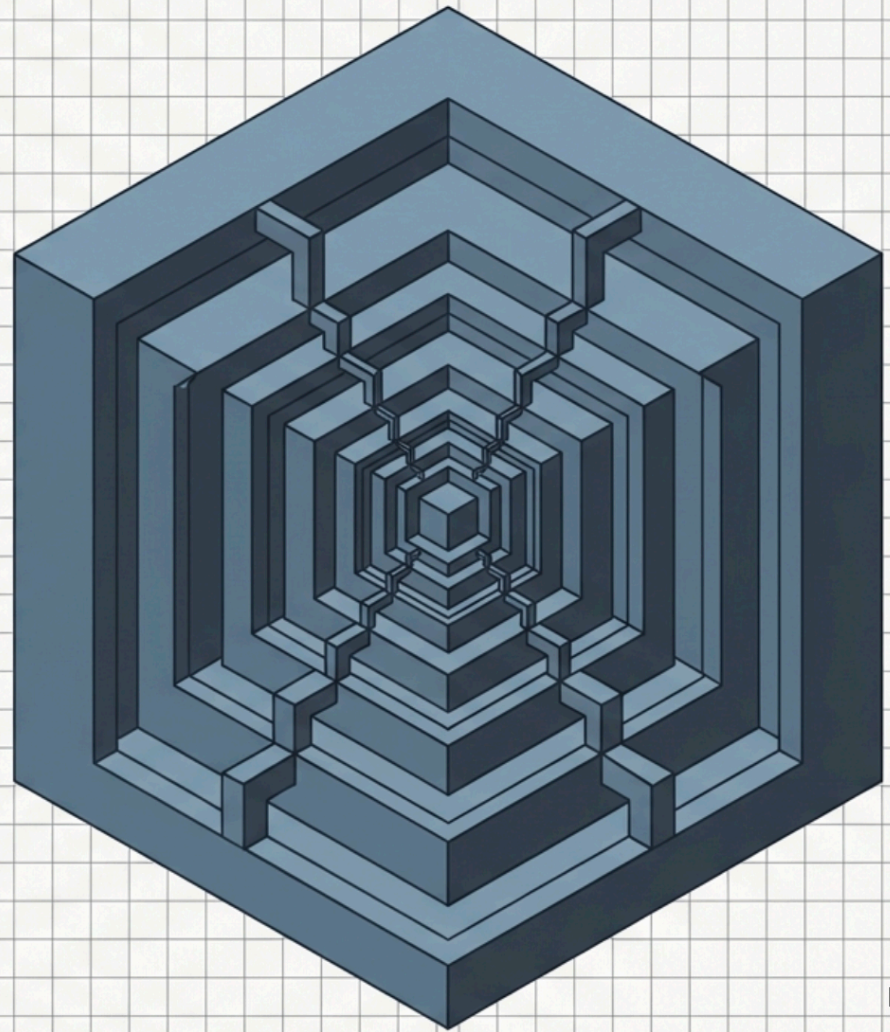


RICORSIONE

in C#



Cos'è la ricorsione?

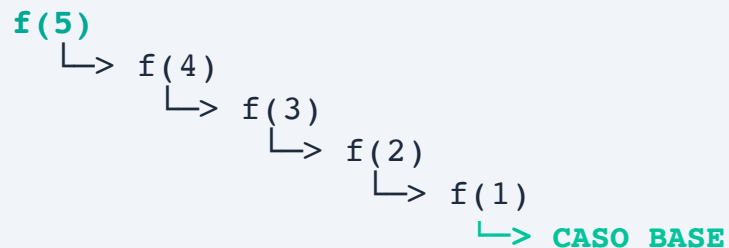
DEFINIZIONE

La ricorsione è una tecnica di programmazione in cui una funzione chiama se stessa per risolvere un problema.

Un problema viene scomposto in sotto-problemi più piccoli e simili fino a raggiungere un caso base risolvibile direttamente.

ESEMPIO VISIVO

Una funzione che chiama se stessa:



Anatomia di una funzione ricorsiva

1. CASO BASE

La condizione di uscita che ferma la ricorsione

2. CASO RICORSIVO

La chiamata della funzione su un problema più piccolo

```
public int Esempio(int n)
{
    if (n <= 0) return 1;
    // ↑ CASO BASE

    return n * Esempio(n - 1);
    // ↑ CASO RICORSIVO
}
```

Esempio 1: Fattoriale

CONCETTO

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

$$5! = 5 \times 4!$$

```
public int Fattoriale(int n)
{
    // Caso base
    if (n <= 1)
        return 1;

    // Caso ricorsivo
    return n * Fattoriale(n - 1);
}
```

TRACCIA DI ESECUZIONE

```
Fattoriale(5)
= 5 × Fattoriale(4)
= 5 × (4 × Fattoriale(3))
= 5 × (4 × (3 × Fattoriale(2)))
= 5 × (4 × (3 × (2 × Fattoriale(1))))
= 5 × (4 × (3 × (2 × 1)))
= 5 × (4 × (3 × 2))
= 5 × (4 × 6)
= 5 × 24
= 120
```


Esempio 2: Fibonacci

SEQUENZA DI FIBONACCI

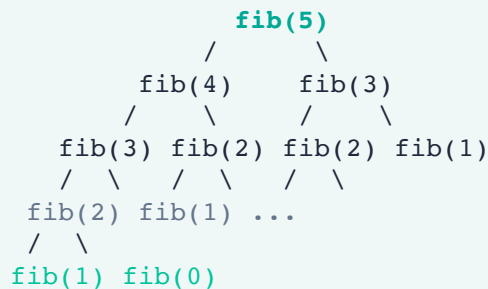
0, 1, 1, 2, 3, 5, 8, 13, 21, 34...

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

```
public int Fibonacci(int n)
{
    // Casi base
    if (n == 0) return 0;
    if (n == 1) return 1;

    // Caso ricorsivo
    return Fibonacci(n - 1) +
           Fibonacci(n - 2);
}
```

ALBERO DELLE CHIAMATE (fib(5))



⚠ **Molte chiamate ripetute!**

Esempio 3: Somma di un array

IDEA RICORSIVA

Somma([1,2,3,4]) = 1 + Somma([2,3,4])

```
public int SommaArray(int[] arr, int i)
{
    // Caso base: fine array
    if (i >= arr.Length)
        return 0;

    // Caso ricorsivo
    return arr[i] + SommaArray(arr, i+1);
}
```

ESEMPIO DI ESECUZIONE

Array: [10, 20, 30, 40]

```
SommaArray(arr, 0)
= 10 + SommaArray(arr, 1)
= 10 + (20 + SommaArray(arr, 2))
= 10 + (20 + (30 + SommaArray(arr, 3)))
= 10 + (20 + (30 + (40 + SommaArray(arr, 4))))
= 10 + (20 + (30 + (40 + 0)))
= 10 + (20 + (30 + 40))
= 10 + (20 + 70)
= 10 + 90
= 100
```

Ricorsione vs Iterazione

VERSIONE RICORSIVA

```
public int Somma(int n)
{
    if (n <= 0) return 0;
    return n + Somma(n - 1);
}
```

VERSIONE ITERATIVA

```
public int Somma(int n)
{
    int somma = 0;
    for (int i = 1; i <= n; i++)
        somma += i;
    return somma;
}
```

CONFRONTO

- ✓ **Ricorsione:** più elegante e intuitiva per problemi con struttura ricorsiva
- ✓ **Iterazione:** più efficiente in termini di memoria e performance

Quando usare la ricorsione?

✓ QUANDO USARLA

- Problemi con struttura ricorsiva naturale
- Alberi e grafi
- Divide et impera
- Backtracking
- Quando il codice risulta più chiaro

✗ QUANDO EVITARLA

- Loop semplici sostituibili con for/while
- Ricorsione molto profonda
- Quando serve massima performance
- Sottoproblemi ripetuti (senza memoization)

Errori comuni e Stack Overflow

ERRORE 1: Mancanza del caso base

```
public int Errore(int n) { return n * Errore(n - 1); }
```

→ La funzione non si ferma mai → `StackOverflowException`!

ERRORE 2: Caso base irraggiungibile

```
public int Errore(int n) { if (n == 0) return 1; return n * Errore(n + 1); }
```

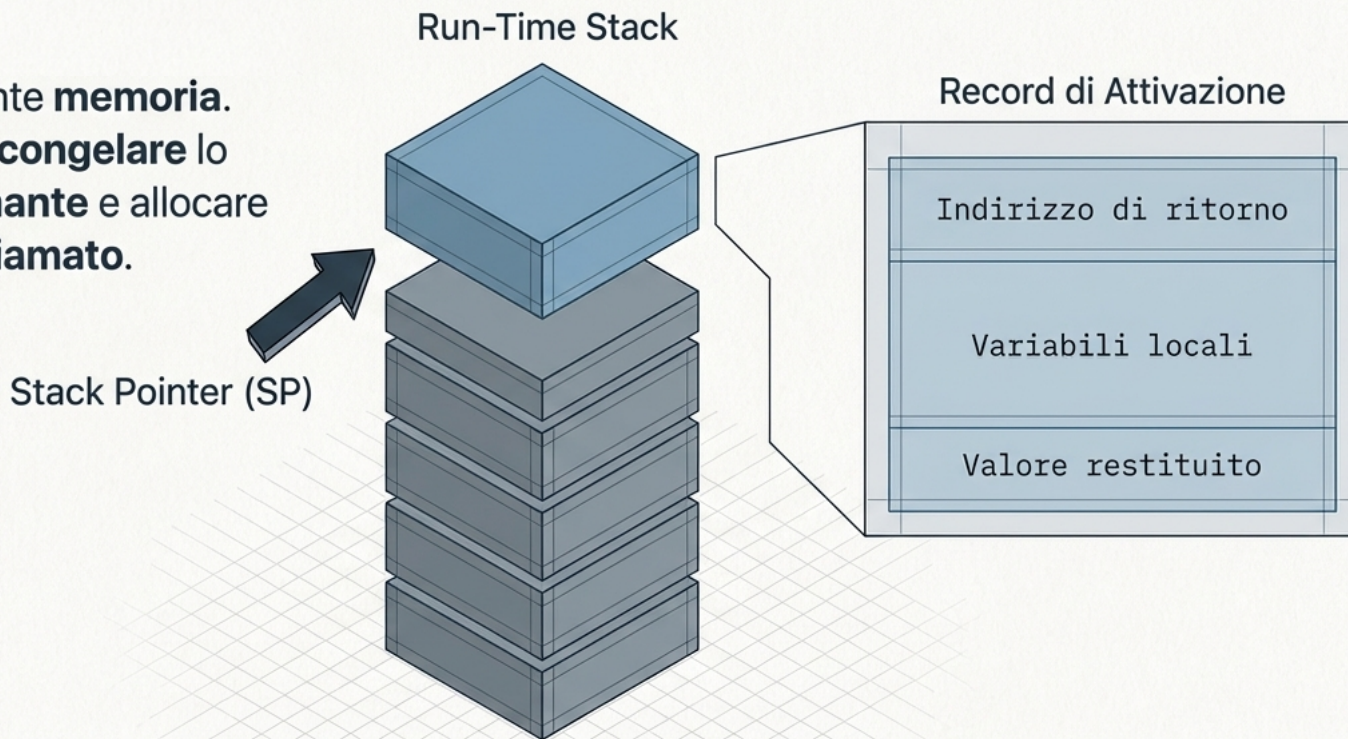
→ Il valore aumenta invece di diminuire!



Stack Overflow: se le chiamate ricorsive sono troppe, la memoria dello stack si esaurisce e il programma va in crash.

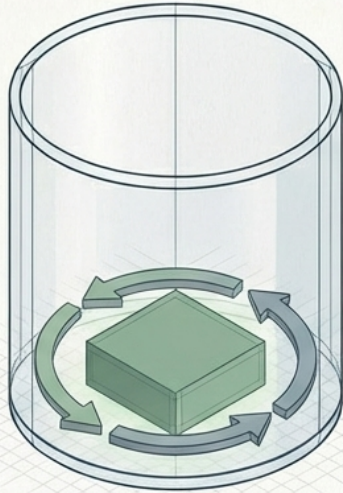
Dietro le Quinte: Il Call Stack

Ogni chiamata
alloca fisicamente **memoria**.
Il sistema deve **congelare** lo
stato del **chiamante** e allocare
spazio per il **chiamato**.



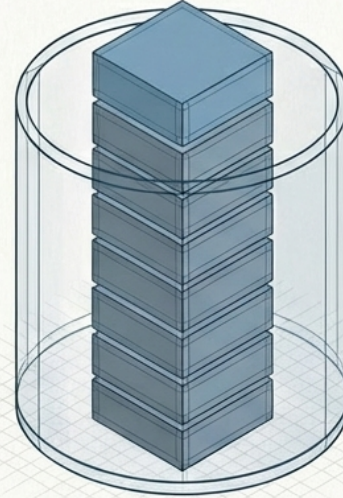
Il Prezzo dell'Eleganza

Iterazione



Space Complexity: $O(1)$
Usa 1 solo Stack Frame.

Ricorsione



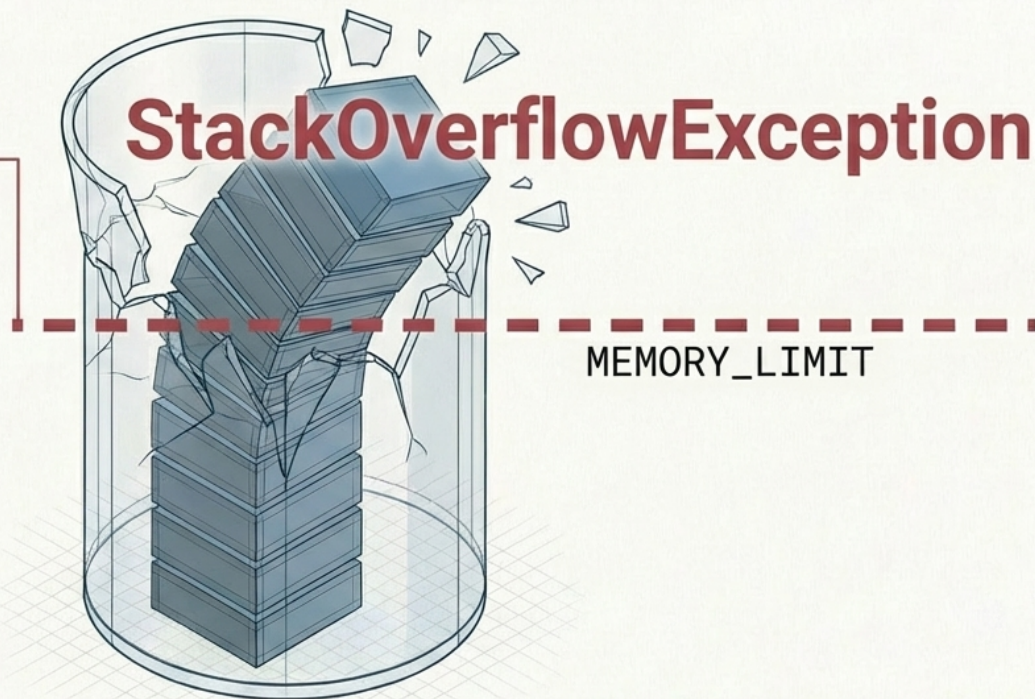
Space Complexity: $O(N)$
Usa N Stack Frames.

Calcolare `factorial(10000)` significa spingere 10.000 frame fisici nella memoria.

Il Punto di Rottura

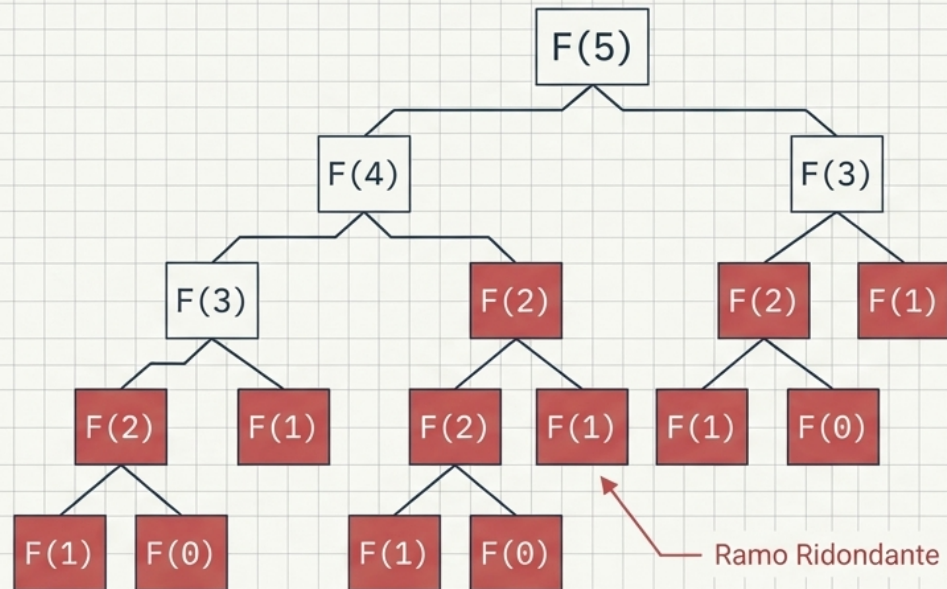
L'iterazione entra in un loop infinito.
La ricorsione si schianta.

A differenza dei cicli `while`, che impegnano la CPU all'infinito, una ricorsione troppo profonda o priva di un Caso Base satura lo spazio spazio fisico allocato al thread. In C#, questo causa il crash irreversibile dell'applicazione.



La Trappola dell'Esplosione Combinatoria

L'inefficienza nasce dalla cieca ripetizione degli stessi calcoli in rami separati dell'albero di esecuzione.



> Calcolo Fib(31)...

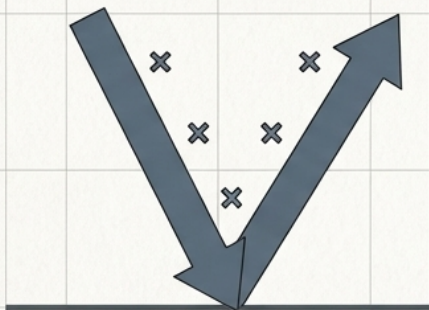
Chiamate generate: 4.356.617

Addizioni eseguite: 2.178.308

Ottimizzazione 1:

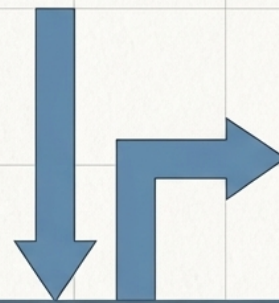
Ricorsione in Coda (Tail Recursion)

Ricorsione Standard



Percorso a 'V': discesa profonda e risalita per completare i calcoli in sospeso.

Ricorsione in Coda



Discesa Diretta: l'accumulatore trasporta il totale e il risultato finale viene restituito istantaneamente.

**Regola: La chiamata ricorsiva DEVE essere l'assoluta ultima istruzione.
Nessuna operazione in sospeso al ritorno.**

Anatomia di un Accumulatore

Standard: Moltiplicazione in sospeso

```
int Fact(int n) {  
    if (n == 0) return 1;  
    return n * Fact(n - 1);  
}
```

Operazione in sospeso

Tail-Recursive: Accumulatore puro

```
int FactTail(int n, int acc = 1) {  
    if (n == 0) return acc;  
    return FactTail(n - 1, n * acc);  
}
```

Nessuna operazione in sospeso



Attenzione: Il compilatore JIT di C# non garantisce sempre la Tail Call Optimization (TCO). In C#, in contesti iper-critici, questa logica viene spesso trasformata esplicitamente in un ciclo `while` per sicurezza assoluta.

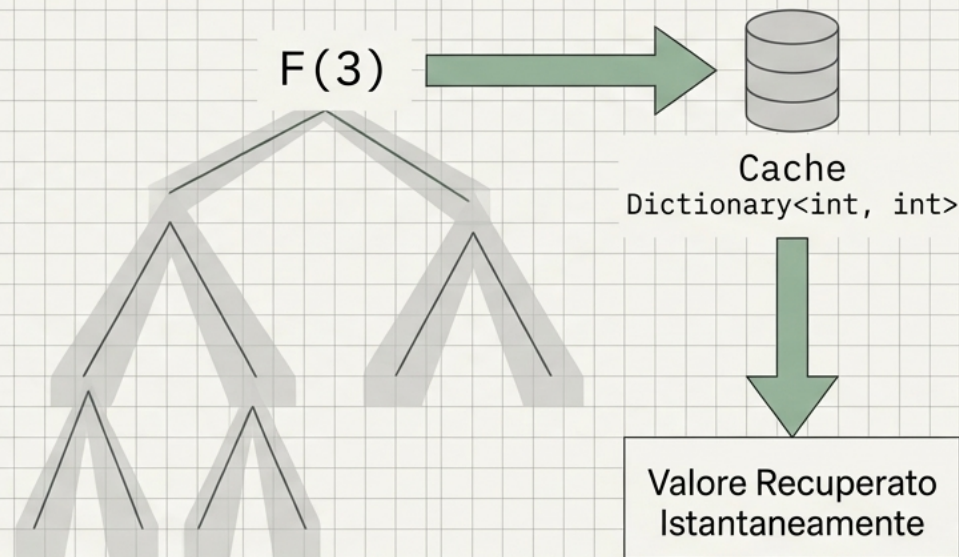
Ottimizzazione 2: Memoization

Salvataggio dello Stato

Conservare i risultati delle chiamate già eseguite in una struttura dati ausiliaria.

Da $O(2^N)$ a $O(N)$

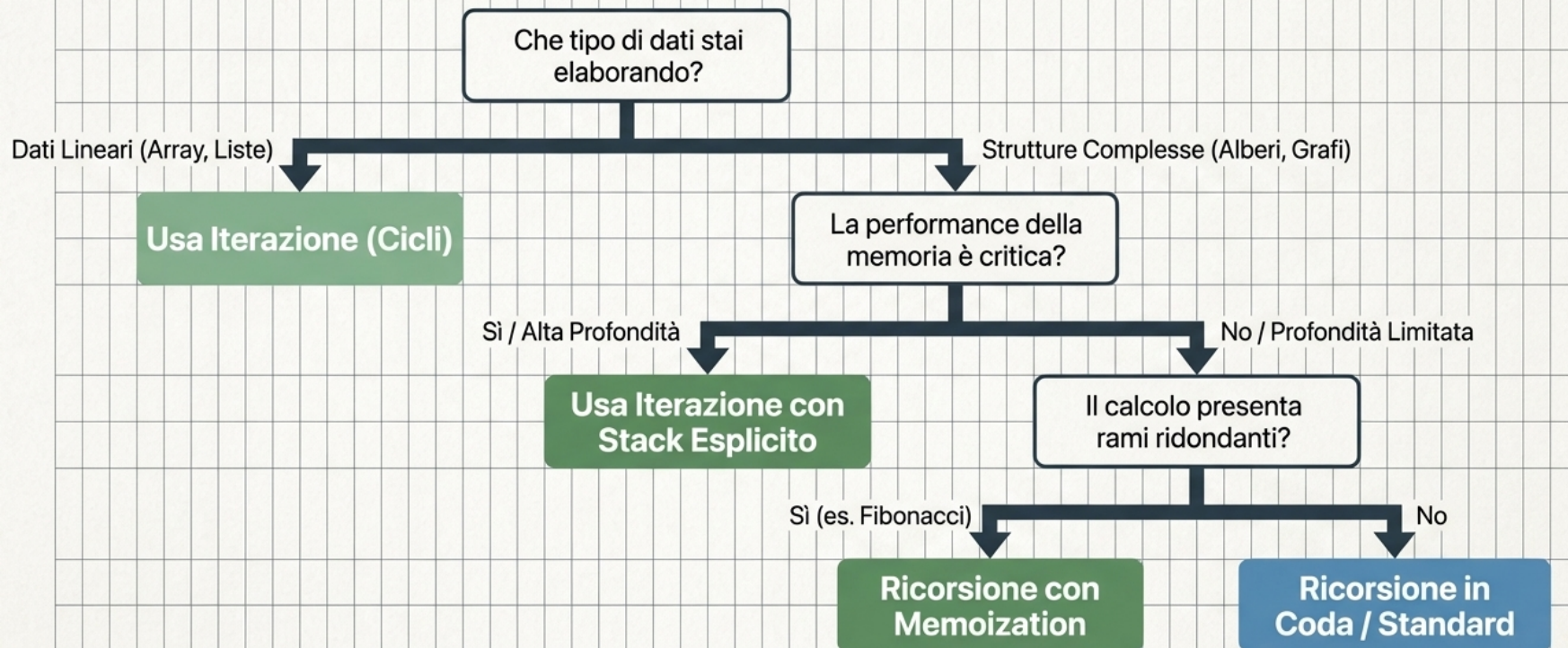
Riduce drasticamente il tempo di calcolo **annullando i rami ridondanti** dell'albero di esecuzione.



Matrice Diagnostica

Parametro	Ricorsione Standard	Ricorsione in Coda	Iterazione (for/while)
Spazio / Memoria	$O(N)$ (Grave)	$O(1)$ (Se ottimizzata)	$O(1)$ (Costante)
Tempo / Overhead	Alto (Creazione Stack)	Medio / Basso	Minimo (Aggiornamento)
Rischio Crash	Elevato (Stack Overflow)	Dipende dal compilatore JIT	Nulla
Leggibilità (Alberi/Grafi)	Eccellente, Intuitiva	Buona	Bassa (Stack esplicito)

L'Albero Decisionale dello Sviluppatore



[1] Il Caso Base è Legge

Definisci e testa sempre la condizione di terminazione prima di scrivere il caso induttivo.

[2] Attenzione ai Limiti C#

Poiché C# non garantisce l'uso dell'opcode `.tail`, converti le ricorsioni in coda in cicli `while` se operi su dataset immensi.

[3] Strutture Dati, non Codice

Usa array di supporto o `Span<T>` invece della ricorsione per invertire stringhe o processare dati lineari.

[4] Memoization Obbligatoria

Se la tua ricorsione non è lineare (chiama se stessa più di una volta per ciclo), usa sempre un dizionario o array di cache.